BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding

https://colab.research.google.com/drive/1Xtcb5tak9Xs0zbO5FrnTc6uFHkzbmhKa?usp=sharing

7.3 BERT

(Bidirectional Encoder Representations from Transformers)

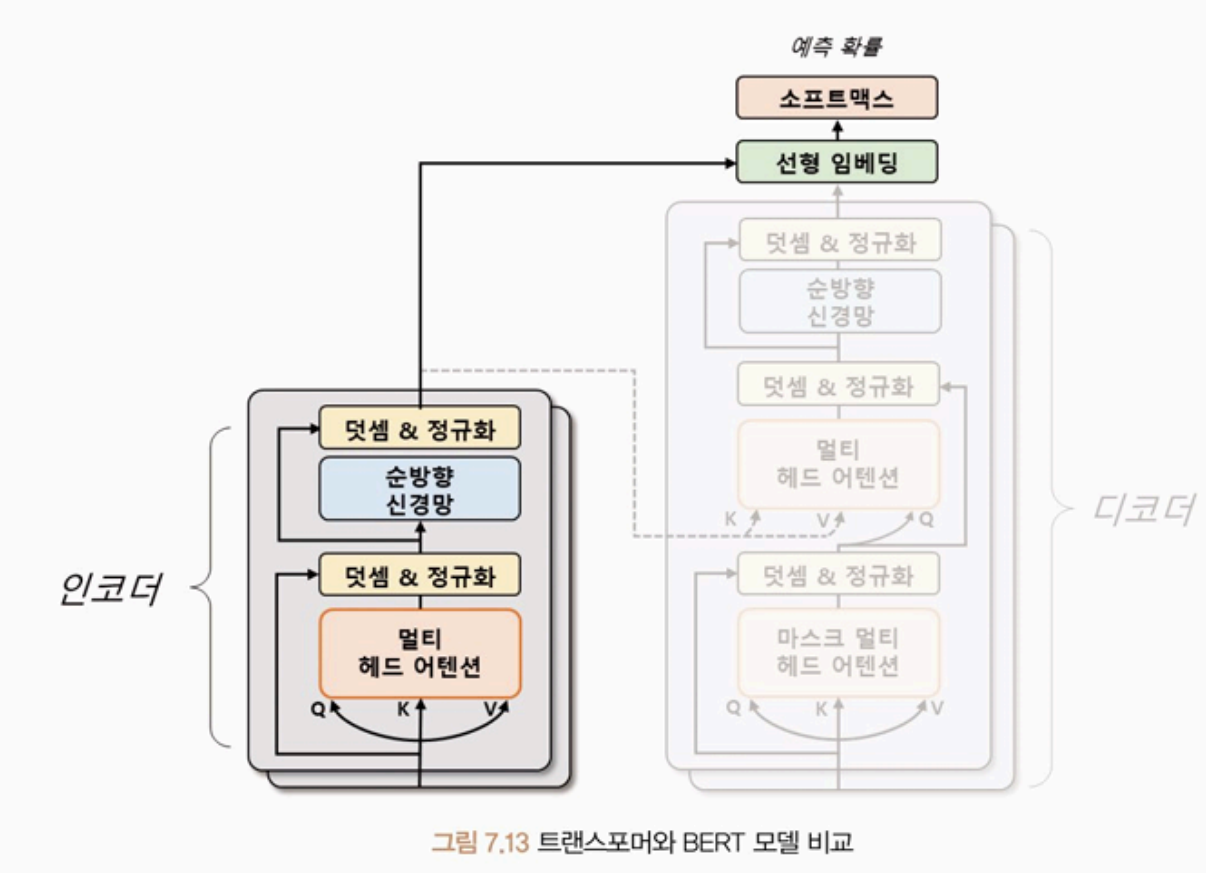


그림 7.13 트랜스포머와 BERT 모델 비교

- 트랜스포머 기반 양방향 인코더 사용 자연어 처리 모델
- 사전학습 후 fine-tuning을 통해 다양한 자연어 처리 작업에 적용가능
  - e.g. 문장 분류, 감성 분석, 질문 응답, 기계 번역 등
  - fine-tuning : 해당 작업에 맞는 계층 추가, 손실 함수 정의해 학습

✅ 양방향 인코더

- 입력 시퀀스를 양쪽 방향에서 처리
  ⇒ 이전 + 이후 단어 모두 참조해 단어의 의미와 문맥 파악
- 기존(좌→우 순차 학습)보다 더 정확히 문맥 파악, 다양한 자연어처리 작업에서 성능 ⬆️
- 입력 문장의 단어를 좌우 양방향으로 처리해 문맥 정보를 모델링 → 인코더 계층만을 이용해 학습

7.3.1 사전 학습 방법

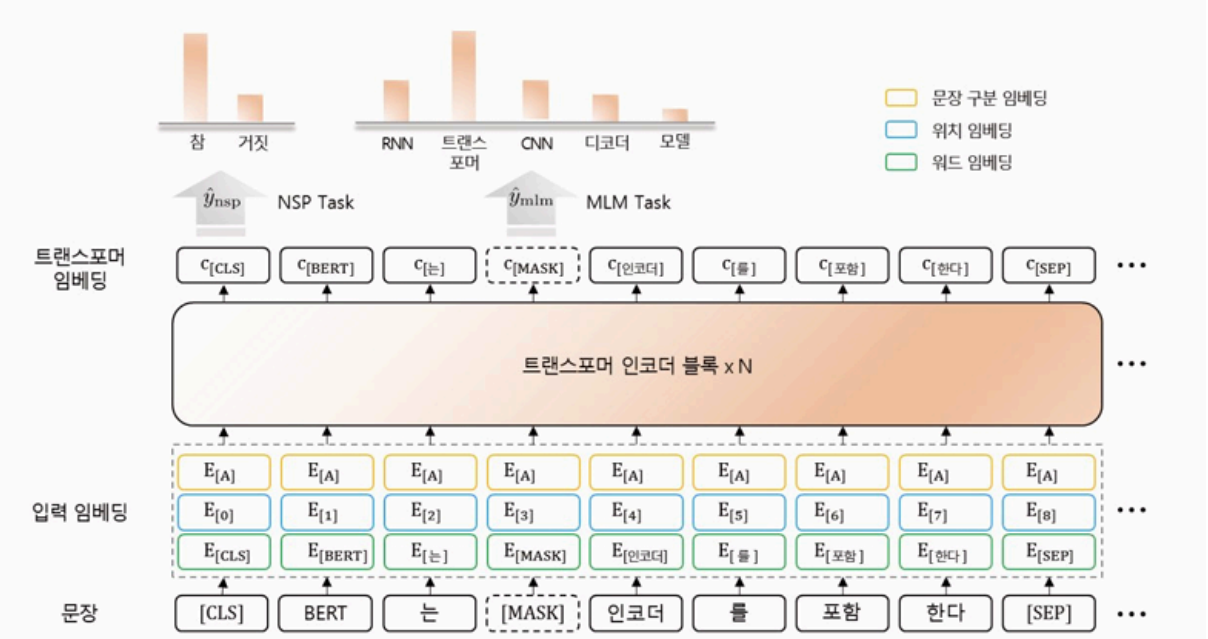


그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

- BERT는 대규모 데이터로 사전학습된 모델 → 전이학습에 주로 이용됨.
- 일부/전체를 다른 작업에서 재사용해 적은 양의 데이터로 높은 정확도 달성 가능

✅ MLM(Masked Language Modeling, 마스킹된 언어 모델링)

- 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어 예측

  ⇒ 누락된 단어를 추론하는 능력 ⬆️
- 이 과정에서 양방향 문맥 정보 참고

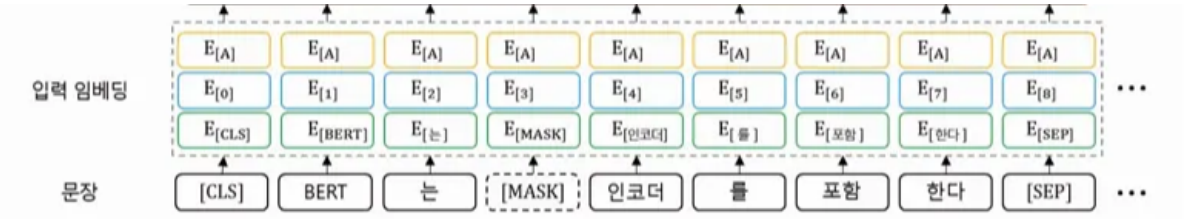
✅ NSP(Next Sentence Prediction, 다음 문장 예측)

- 2번째 문장이 1번째 문장의 다음에 오는 문장인지 여부 판단
- 두 문장이 연속적 관계? 아니야? 판단

⇒ 두 문장 간의 관계 학습 + 문장 간의 의미적인 유사성 파악

✅ 토큰 추가

- 입력 문장에 특수한 토큰을 추가해 모델 학습&추론 과정에서 필요한 정보 제공
  - CLS 토큰 - 입력 문장 시작 부분에 추가,
    문장 분류 작업에 사용 / 어떤 유형의 문장인지 미리 파악 가능
  - SEP 토큰 - 입력 문장 내에서 두개 이상 문장 구분
  - MASK 토큰 - 입력 문장 내 임의로 선택된 단어를 가리키는 특별한 토큰, 마스킹에 사용



Introduction (Background, Motivation)

Background

- 기존 NLP 사전학습 방법은 크게 **feature-based 방식**과 **fine-tuning 방식**으로 나뉘었습니다.
- ELMo는 양방향 정보를 쓰긴 했지만, **좌→우 / 우→좌를 각각 따로 학습한 뒤 결합**하는 구조였습니다.
- GPT는 Transformer 기반 fine-tuning 방식이었지만, **left-to-right** 구조라 오른쪽 문맥을 직접 보지 못했습니다.
- 따라서 문장 전체 의미를 동시에 이해해야 하는 과제에서는 구조적 한계가 있었습니다.

Motivation

- 저자들은 **깊은 양방향 문맥 표현(deep bidirectional representation)** 을 사전학습하는 것이 중요하다고 보았습니다.
- 일반 language model 방식으로는 양방향 학습이 어렵기 때문에, 새로운 학습 목표가 필요했습니다.
- 이를 해결하기 위해 **Masked Language Model(MLM)** 과 **Next Sentence Prediction(NSP)** 을 제안했습니다.

Related Work

ELMo와의 차이

- ELMo는 양방향처럼 보이지만, 실제로는 **두 개의 단방향 LM을 결합**한 구조입니다.
- downstream task에서는 주로 feature로 주입되었습니다.
- BERT는 표현만 주는 것이 아니라, **모델 전체를 fine-tuning** 합니다.

GPT와의 차이

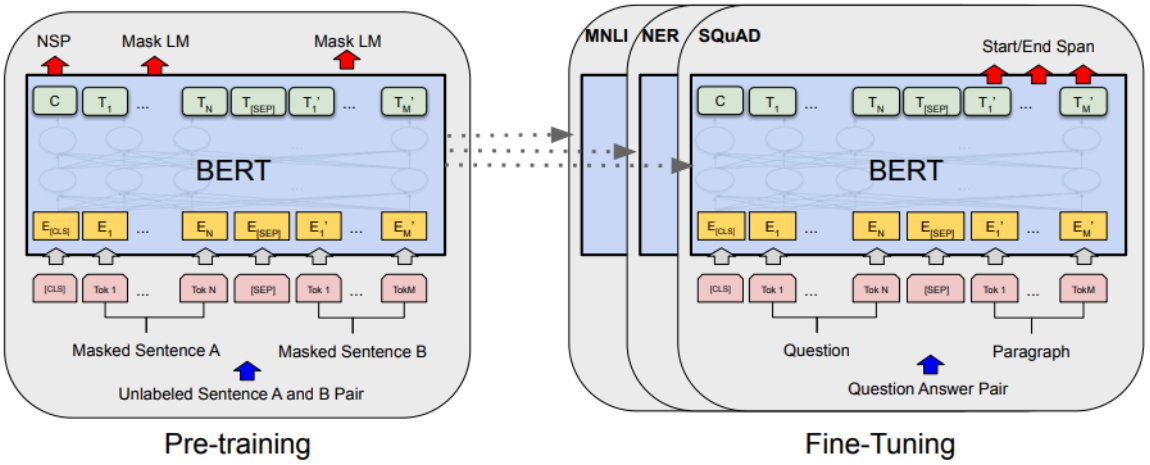
- GPT는 Transformer 기반이지만 **left-to-right** 제약이 있습니다.
- BERT는 **bidirectional self-attention** 을 사용합니다.
- 따라서 BERT는 각 단어를 볼 때 **좌우 문맥을 동시에 반영**할 수 있습니다.

기존 전이학습과의 차이

- 기존에는 라벨이 있는 큰 데이터셋에 먼저 학습한 뒤 전이하는 방식이 많았습니다.
- BERT는 **라벨 없는 대규모 텍스트에서 먼저 일반 언어 이해 능력을 학습**합니다.
- 이후 적은 라벨만으로도 다양한 과제에서 성능을 냅니다.

Method (Architecture, Main Idea)

Architecture



- BERT의 **pre-training** → **fine-tuning** 전체 흐름을 보여줍니다.

- pre-training에서는 MLM과 NSP를 동시에 수행합니다.
- fine-tuning에서는 같은 BERT 본체 위에 task별 얇은 head만 추가합니다.
- **하나의 사전학습 모델을 여러 과제에 재사용**
- BERT는 **Transformer encoder-only** 구조입니다.
- 두 가지 주요 모델이 있습니다.
  - **BERT_BASE**
    - Layer 수 L=12
    - Hidden size H=768
    - Attention head 수 A=12
    - 파라미터 약 110M 개
  - **BERT_LARGE**
    - Layer 수 L=24
    - Hidden size H=1024
    - Attention head 수 A=16
    - 파라미터 약 340M

Input Representation

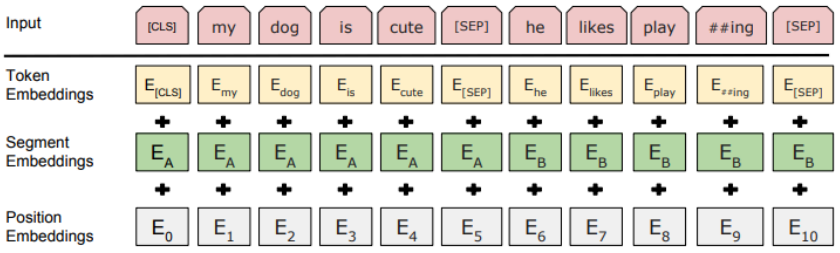


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

- 입력 임베딩 = **token + segment + position embedding** 의 합
- 문장 A/B 구분이 segment embedding으로 들어갑니다.
- BERT가 문장쌍 task를 잘 다루는 이유를 설명

- 문장쌍 입력 형식은 다음과 같습니다.
- `[CLS] sentence A [SEP] sentence B [SEP]`
- `[CLS]` 는 문장 전체 표현에 사용
- `[SEP]` 는 문장 경계를 구분

Main Idea

- 일반 LM은 다음 단어를 예측하기 때문에 양방향 문맥을 동시에 쓰기 어렵습니다.
- BERT는 일부 단어를 가리고 맞히게 하는 **MLM**으로 이를 해결했습니다.
- 문장 간 관계를 학습하기 위해 **NSP**를 추가했습니다.

📒 ✅ $C \in RH$

- `[CLS]` 토큰의 최종 hidden state입니다.
- 문장 전체를 대표하는 벡터로 사용됩니다.
- 문장 분류, 문장쌍 분류 등에 활용됩니다.

✅ $Ti \in RH$

- i번째 토큰의 최종 hidden state입니다.
- 각 토큰의 문맥적 표현입니다.
- NER, QA 같은 token-level task에 사용

📒 **분류식**

- 문장 분류는 `[CLS]` 벡터 C에 선형층 W를 곱하고 softmax를 취합니다.
- 형태는 `log(softmax(CW^T))` 입니다.
- **문장 전체 표현 ⇒ 클래스 확률(softmax)**

✅ **QA 시작 위치 확률**

- 시작 벡터 S를 둡니다.
- 각 토큰이 시작점일 확률
  $P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$
- $S \cdot T_i$ 는 i번째 토큰이 시작점으로 적절한 정도

✅ **QA 끝 위치 확률**

- 끝 벡터 E를 둡니다.
- 마찬가지 방식으로 끝 위치 확률을 계산합니다.

✅ **span score-** 후보 답변 $span(i, j)$ 의 점수
$S \cdot T_i + E \cdot T_j$

- 시작점 적합도와 끝점 적합도를 더한 값입니다.
- 이 점수가 가장 높은 구간을 정답으로 선택합니다.

✅ **no-answer 점수**
$s_{\text{null}} = S \cdot C + E \cdot C$

- SQuAD 2.0에서는 `[CLS]` 를 no-answer의 기준으로 사용합니다.
- 일반 span 최고점과 비교해서 no-answer 여부를 결정합니다.

Pre-training Task 1: MLM

- 전체 토큰 중 **15%를 예측 대상**으로 선택합니다.
- 선택된 토큰 처리 방식은 다음과 같습니다.
- **80%**: `[MASK]` 로 바꿈
- **10%**: 랜덤 토큰으로 바꿈
- **10%**: 그대로 둠

- 목적은 masked position의 원래 단어를 맞히는 것입니다.
- 이렇게 하면 각 토큰 표현이 **좌우 문맥 모두를 활용**하게 됩니다.

Pre-training Task 2: NSP

- 데이터 구성은 다음과 같습니다.
- **50%**: 실제 다음 문장 → IsNext
- **50%**: 랜덤 문장 → NotNext
- `[CLS]` 벡터를 이용해 분류합니다.
- 문장쌍 관계를 이해하는 데 도움을 주도록 설계되었습니다.

Datasets

- **BooksCorpus**: 약 8억 단어
- **English Wikipedia**: 약 25억 단어
- 총 약 **33억 단어** 규모입니다.

Training Setting

- 학습 step 수: **1M steps**
- batch 크기: **128,000 tokens**
- optimizer: **Adam**
- learning rate: **1e-4**
- warmup: **10,000 steps**
- weight decay: **0.01**
- dropout: **0.1**
- activation: **GELU**
- 학습의 90%는 길이 128, 마지막 10%는 길이 512 시퀀스를 사용했습니다.
- 문장 A 뒤에 오는 문장 B가 실제 다음 문장인지 맞히는 task입니다.

Experiments

GLUE

- BERT_LARGE는 GLUE 평균 점수 **82.1**을 기록했습니다.
- 당시 OpenAI GPT의 **75.1**보다 높았습니다.
- MNLI, SST-2, MRPC 등 다양한 과제에서 강한 성능을 보였습니다.

SQuAD

- SQuAD v1.1에서 매우 높은 F1을 기록했습니다.
- SQuAD v2.0에서도 no-answer를 포함한 설정에서 강한 성능을 보였습니다.
- 질문응답 task에서 BERT의 양방향성이 특히 효과적임을 보여줍니다.

SWAG

- SWAG에서도 GPT보다 높은 accuracy를 보였습니다.
- 문장쌍 이해나 문맥적 추론에 강하다는 점을 보여줍니다.

Ablation Study

- **NSP 제거** 시 성능이 하락했습니다.
- **LTR + No NSP** 설정에서는 성능이 더 크게 떨어졌습니다.
- 특히 QA와 문장쌍 과제에서 양방향 구조의 중요성이 드러났습니다.

모델 크기 효과

- BERT_LARGE가 BERT_BASE보다 전반적으로 더 높은 성능을 보였습니다.
- 즉, 모델을 크게 키우는 것이 실제 downstream 성능 향상으로 이어졌습니다.

Conclusion (Contribution, Future Work)

Contribution

- **양방향 Transformer encoder 기반 사전학습**을 성공적으로 제시했습니다.
- **MLM + NSP** 라는 새로운 pre-training objective를 도입했습니다.
- 복잡한 task-specific 구조 없이도 strong baseline을 넘어서는 성능을 달성했습니다.
- 현대 NLP의 **pre-train + fine-tune 패러다임**을 정착시킨 대표 논문입니다.

Future Work

- NSP가 정말 필수적인가?
- MLM보다 더 효율적인 objective는 없는가?
- 더 긴 문서를 효율적으로 처리할 수 있는가?
- encoder-only를 넘어 생성형 모델과 어떻게 연결할 것인가?